UNIVERSITÉ JOSEPH KI-ZERBO

(UJKZ)

Institut Burkinabè des Arts et Métiers

(IBAM)

Master 1 Informatique / SI

Unité d'enseignement : Virtualisation et Cloud Computing

Module DevOps et infrastructures reproductibles

GUIDE DÉTAILLÉ DE DÉMONSTRATION

RÉSULTATS-BF

Infrastructure résiliente et reproductible avec Vagrant et VirtualBox

GROUPE 10

Membres du groupe — M1/SI :

CONGO Céphas

LOMPO Ephraim

BOUGOUMA Leïla

KAZIENGA Eva Benjamine

Enseignant :

Dr SOMDA Flavien Hervé

Année académique 2026–2027

Table des matières

1. Ce que la présentatrice doit comprendre avant de commencer **4**

1.1. Objectif du projet . . . 4

1.2. Architecture en une image . . . 4

1.3. Vocabulaire simple . . . 4

2. Les membres et leurs contributions réelles **6**

3. Fichiers importants à savoir expliquer **7**

4. Préparation avant l'enregistrement **8**

4.1. Choisir la présentatrice . . . 8

4.2. Préparer l'écran . . . 8

4.3. Contrôles techniques obligatoires . . . 8

5. Plan conseillé d'une vidéo de vingt minutes **9**

6. Script complet à suivre pendant la vidéo **10**

Étape 1 — introduction . . . 10

Étape 2 — montrer les contributions . . . 10

Étape 3 — montrer l'inventaire . . . 10

Étape 4 — destruction et reconstruction . . . 10

Étape 5 — expliquer l'architecture pendant l'attente . . . 11

Étape 6 — vérifier la reconstruction . . . 11

7. Trois scénarios recommandés pour la vidéo **12**

Scénario 1 — V1, répartition des requêtes . . . 12

Scénario 2 — V2, panne d'une application . . . 12

Scénario 3 — R5, réplication PostgreSQL . . . 12

8. Autres scénarios à connaître **13**

V3 — montée en charge . . . 13

R2 — idempotence . . . 13

E3 — cache Redis . . . 13

E4 — déploiement sans coupure . . . 13

V4 — panne de la base principale . . . 13

V5 — reconstruction complète . . . 13

V6 — répartiteur redondant . . . 14

9. Preuves enregistrées **15**

10. Conclusion prête à prononcer **16**

11. Questions probables et réponses simples **17**

Pourquoi quatre applications ? . . . 17

Pourquoi /sante interroge-t-elle PostgreSQL ? . . . 17

Pourquoi bdd2 est-elle normalement en lecture seule ? . . . 17

Pourquoi la promotion n'est-elle pas automatique ? . . . 17

Redis contient-il les résultats officiels ? . . . 17

Pourquoi les performances baissent-elles avec quatre applications ? . . . 17

Comment les secrets sont-ils protégés ? . . . 17

Pourquoi V6 n'est-elle pas présentée ? . . . 17

Est-ce que le projet fonctionne dans la VM Ubuntu ouverte manuellement ? . . . 17

12. Incidents possibles pendant la répétition **18**

scripts/ops.py est introuvable . . . 18

Vagrant indique qu'une VM n'est pas démarrée . . . 18

/sante répond 503 . . . 18

V1 échoue . . . 18

Le terminal affiche ^[[200~ . . . 18

/dev/vboxdrv est absent . . . 18

13. Ordre de sécurité à mémoriser **19**

14. Liste finale avant de publier la vidéo **20**

Ce document permet à une personne qui n'a pas développé le projet de présenter RESULTATS-BF correctement. Il contient le texte à prononcer, les commandes à exécuter, les résultats attendus et les réponses aux questions probables.

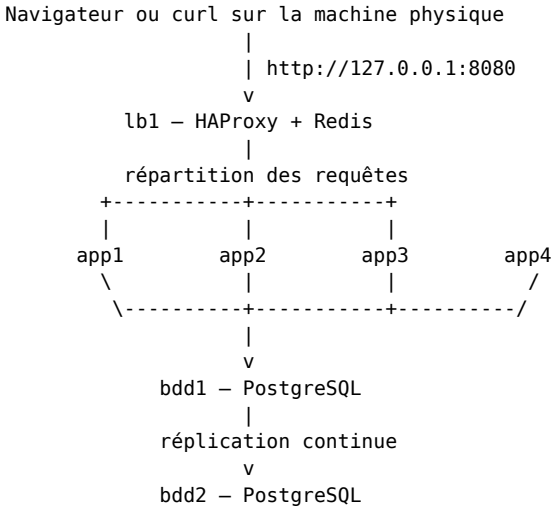
La démonstration se fait depuis la **machine Linux physique**, dans le dossier du projet. Il ne faut pas ouvrir une VM Ubuntu et y recopier les commandes : Vagrant et VirtualBox créent et administrent eux-mêmes les sept machines virtuelles.

1. Ce que la présentatrice doit comprendre avant de commencer

1.1. Objectif du projet

RESULTATS-BF simule une plateforme de consultation de résultats d'examen. Elle doit continuer à répondre pendant un pic de demandes et pendant la panne d'un serveur applicatif. Les données sont stockées dans PostgreSQL et répliquées sur une seconde base. Toute l'infrastructure est créée automatiquement avec Vagrant.

1.2. Architecture en une image



Les sept VM sont donc :

VM	Adresse privée	Fonction
lb1	192.168.56.11	HAProxy répartit les requêtes ; Redis sert de cache
app1	192.168.56.21	API Flask sans état
app2	192.168.56.22	API Flask sans état
app3	192.168.56.23	API Flask sans état
app4	192.168.56.24	API Flask sans état
bdd1	192.168.56.31	base PostgreSQL principale, accessible en écriture
bdd2	192.168.56.32	réplique PostgreSQL, normalement en lecture seule

Le seul port applicatif exposé sur la machine physique est le port `8080`. Les ports SSH redirigés sur `127.0.0.1` servent uniquement à l'administration par Vagrant.

1.3. Vocabulaire simple

- **Hôte** : la machine physique sur laquelle VirtualBox est installé.
- **VM** : ordinateur virtuel créé dans VirtualBox.

- **Vagrant** : outil qui crée, démarre et configure les VM à partir du code.
- **HAProxy** : répartiteur qui envoie les requêtes vers les quatre applications.
- **Application sans état** : aucune donnée importante n'est conservée dans `app1` à `app4`. Une application peut être recréée sans perte métier.
- **PostgreSQL** : base contenant les résultats durables.
- **Réplication** : copie continue des changements de `bdd1` vers `bdd2`.
- **Promotion** : transformation volontaire de `bdd2` en nouvelle base principale après l'arrêt de `bdd1`.
- **Redis** : cache temporaire qui accélère les consultations répétées.
- **Idempotence** : appliquer deux fois la même configuration ne doit pas produire un nouveau changement lors de la seconde application.
- **Déploiement progressif** : mise à jour des applications une par une, sans interrompre le service.

2. Les membres et leurs contributions réelles

Une seule personne présente la vidéo, mais elle doit annoncer les quatre membres et montrer que leurs contributions existent réellement dans Git.

Membre	Contribution principale	Commits à connaître
Céphas CONGO	architecture, intégration initiale, preuves et fusion des PR	`56e79f3` et commits de fusion
Lompo Ephraim	CI GitHub Actions et détection des conflits d'adresses VirtualBox	`bd482e4`, `0e1ff83`
Leïla BOUGOUMA	erreurs JSON de l'API et contrôle de la temporisation HAProxy	`00ff746`, `118c64e`, `cd63415`
Eva Benjamine Kazienga	données, réplication, diagnostic et réponse 503 de HAProxy	`2492ee1`, `452b7bf`

Commande à montrer sans afficher les adresses électroniques :

```
git log --all --no-merges --format='%h | %an | %s'
```

Texte conseillé :

> *Le projet a été réalisé par les quatre membres du groupe. Céphas a assuré l'architecture et l'intégration, Lompo la validation continue et le réseau, Leïla la partie applicative, et Eva la partie données et résilience. Les contributions sont visibles dans les commits et les Pull Requests du dépôt. Je réalise aujourd'hui la démonstration au nom de tout le groupe.*

3. Fichiers importants à savoir expliquer

Fichier ou dossier	Explication à donner
`Vagrantfile`	décrit la création des VM, leurs réseaux, leur mémoire et leur provisionnement
`config/topology.rb`	source unique des noms, rôles et adresses IP
`app/service.py`	API Flask avec `/info`, `/sante` et `/resultat`
`scripts/provision.py`	installe et configure chaque VM selon son rôle
`scripts/ops.py`	exécute V1, R2, R5, le cache, la promotion et les mesures
`scripts/validate-live.py`	teste la panne applicative V2 et le déploiement E4
`scripts/validate-recovery.py`	teste la panne de la base V4 puis la reconstruction V5
`results/`	contient les preuves JSON, CSV et ApacheBench horodatées
`docs/`	contient le rapport, le journal, les exigences et les guides
`.env`	contient les secrets locaux ; il est ignoré par Git et ne doit jamais être affiché
`.runtime/`	état d'exécution généré localement ; il ne doit pas être publié
`.vagrant/`	identifiants locaux des VM ; ce dossier ne contient pas le `Vagrantfile`

Les trois routes de l'application :

- `/info` indique le nom du serveur applicatif et la version déployée ;
- `/sante` vérifie qu'une base accessible en écriture répond réellement ;
- `/resultat?matricule=000001` renvoie le résultat d'un candidat en JSON.

4. Préparation avant l'enregistrement

4.1. Choisir la présentatrice

Nom de la présentatrice : _____

Elle doit lire ce guide une fois entièrement, puis effectuer au moins une répétition sans enregistrement. Un autre membre garde ce document ouvert sur un second appareil et suit la liste des étapes.

4.2. Préparer l'écran

1. Fermer les notifications, la messagerie et les fenêtres personnelles.
2. Ouvrir deux terminaux dans le dossier du projet.
3. Agrandir la police du terminal.
4. Ouvrir ce guide sur un second appareil si possible.
5. Ne jamais ouvrir `.env`, `.runtime` ou un ancien message contenant un jeton.
6. Brancher l'ordinateur au secteur et fermer les applications lourdes.
7. Vérifier qu'au moins 5 Gio de mémoire restent disponibles.

Dans les deux terminaux :

```
cd /home/aiobi5/Documents/Codex/2026-10-18/dis/outputs/resultats-bf
```

Le **terminal A** servira à la reconstruction. Le **terminal B** servira à montrer l'architecture, Git et les résultats pendant que la reconstruction travaille.

4.3. Contrôles techniques obligatoires

Avant de démarrer l'enregistrement :

```
ls -l /dev/vboxdrv
VBoxManage --version
vagrant validate
python3 scripts/ops.py preflight
python3 -m unittest discover -s tests -v
```

Résultats attendus :

- `/dev/vboxdrv` existe ;
- VirtualBox affiche une version sans erreur bloquante ;
- Vagrant indique que le fichier est valide ;
- `preflight` trouve `vagrant`, `VBoxManage`, `ruby`, `python3` et `ab` ;
- aucun conflit d'adresse VirtualBox n'est signalé ;
- la suite découvre 42 tests ; sur l'hôte sans binaire HAProxy, 41 réussissent

et le contrôle direct de HAProxy peut être ignoré.

Si `/dev/vboxdrv` est absent, arrêter la préparation et exécuter sur l'hôte :

```
sudo /sbin/vboxconfig
```

Puis redémarrer les contrôles. Il ne faut pas commencer la vidéo avec VirtualBox en erreur.

Vérifier l'inventaire sans afficher les secrets :

```
ruby config/topology.rb
```

La sortie doit contenir `bdd1`, `bdd2`, `app1`, `app2`, `app3`, `app4` et `lb1`.

5. Plan conseillé d'une vidéo de vingt minutes

Temps final	Contenu
0:00–1:00	présentation du groupe et objectif
1:00–2:30	contributions Git et structure du dépôt
2:30–4:30	destruction, lancement de la reconstruction et architecture
4:30–7:00	explication de Vagrant, HAProxy, API, PostgreSQL et Redis
7:00–9:00	fin de reconstruction, état des sept VM et routes HTTP
9:00–11:00	V1 : répartition entre quatre applications
11:00–14:00	V2 : panne d'une application
14:00–16:00	R5 : réplication PostgreSQL
16:00–18:30	E3, E4 et preuves enregistrées
18:30–20:00	limites, conclusion et dépôt GitHub

La reconstruction réelle a déjà demandé 17 min 22 s. Pour une vidéo, conserver le début et la fin de l'exécution, puis accélérer uniquement la période d'installation au montage. Le chronomètre et le résultat final doivent rester visibles. Pour une démonstration réellement en direct, parler de l'architecture pendant l'exécution dans le terminal A.

6. Script complet à suivre pendant la vidéo

Étape 1 — introduction

À l'écran : page principale du dépôt ou terminal placé à la racine.

Texte à prononcer :

> *Bonjour. Nous sommes le groupe 10, composé de Céphas CONGO, Lompo Ephraim, Leïla BOUGOUMA et Eva Benjamine Kazienga. Je vais présenter RESULTATS-BF au nom du groupe. Il s'agit d'une plateforme de consultation de résultats d'examen déployée automatiquement dans VirtualBox avec Vagrant. Notre objectif est de répartir les demandes, supporter la panne d'une application, répliquer la base de données et reconstruire toute la plateforme sans configuration manuelle dans les machines virtuelles.*

Étape 2 — montrer les contributions

Commande :

```
git log --all --no-merges --format='%h | %an | %s'
```

Texte à prononcer :

> *L'historique montre les contributions réelles des quatre membres. Les branches ont été proposées par Pull Requests puis fusionnées dans* `main`*. Chaque contribution correspond à une modification identifiable du réseau, de l'application, des données ou des tests.*

Ne pas expliquer chaque commit pendant plusieurs minutes. Montrer simplement les quatre noms et citer une contribution par personne.

Étape 3 — montrer l'inventaire

Commande :

```
ruby config/topology.rb
```

Texte à prononcer :

> *Cette sortie est générée par notre source unique de topologie. Nous utilisons une base principale, une réplique, quatre applications et un répartiteur. Le réseau provisoire du groupe 10 est* `192.168.56.0/24`*. Modifier le nombre d'applications dans la configuration modifie aussi l'inventaire et la configuration HAProxy ; nous évitons ainsi une liste écrite manuellement à plusieurs endroits.*

Étape 4 — destruction et reconstruction

Dans le **terminal A**, montrer d'abord l'état existant :

```
vagrant status
```

Puis lancer :

```
vagrant destroy -f
time bash scripts/start.sh
```

`scripts/start.sh` :

1. conserve le `.env` existant ou crée des secrets s'il manque ;
2. vérifie les prérequis et les conflits d'adresses ;
3. valide le `Vagrantfile` ;
4. crée et configure les sept VM ;
5. exécute automatiquement V1 à la fin.

Texte à prononcer pendant le lancement :

Je détruis uniquement les VM de ce projet, puis je lance le script de démarrage. Toutes les commandes sont exécutées depuis la machine physique. Vagrant crée les VM et envoie à chacune l'inventaire, les secrets nécessaires à son rôle, le script de provisionnement et le service applicatif. Les dossiers partagés sont désactivés afin que les VM ne dépendent pas du dossier du poste après leur configuration.

Étape 5 — expliquer l'architecture pendant l'attente

Dans le **terminal B**, montrer brièvement les fichiers sans les modifier :

```
sed -n '1,180p' Vagrantfile
sed -n '1,180p' app/service.py
```

Texte à prononcer :

Le client contacte uniquement HAProxy sur le port 8080. HAProxy vérifie la route `/sante` *de chaque application et répartit les requêtes entre les serveurs sains. Les quatre applications utilisent le même code et ne gardent aucune session ni donnée métier locale. Elles peuvent donc être arrêtées et recréées. PostgreSQL conserve les données durables sur bdd1 et copie les changements vers bdd2. Redis accélère les lectures répétées, mais il ne remplace jamais PostgreSQL.*

Puis expliquer les routes :

`/info` *permet d'identifier le serveur qui a répondu.* `/sante` *exécute une requête réelle vers une base accessible en écriture.* `/resultat` *valide un matricule de six chiffres, consulte Redis si le cache est actif, puis PostgreSQL si nécessaire. Les erreurs sont renvoyées en JSON avec un code HTTP adapté.*

Étape 6 — vérifier la reconstruction

Lorsque `scripts/start.sh` est terminé, exécuter :

```
vagrant status
curl -i http://127.0.0.1:8080/sante
curl http://127.0.0.1:8080/info
curl 'http://127.0.0.1:8080/resultat?matricule=000001'
```

Résultats à repérer :

- les sept VM sont `running` ;
- `/sante` répond `HTTP/1.1 200` et `{"status":"ok"}` ;
- `/info` renvoie un `hostname` parmi `app1` à `app4` et une `release` ;
- `/resultat` renvoie les champs `matricule`, `nom`, `serie`, `moyenne` et

`admis`.

Texte à prononcer :

Les sept machines sont actives. La santé publique répond 200 parce qu'au moins une application saine accède à la base principale en écriture. La route `/info` *prouve que la requête traverse HAProxy. La consultation renvoie un document JSON produit à partir des données PostgreSQL, éventuellement servi depuis Redis après la première lecture.*

7. Trois scénarios recommandés pour la vidéo

Pour une vidéo simple et sûre, utiliser **V1, V2 et R5**. Ils montrent la répartition, la tolérance à une panne applicative et la réplication des données, sans changer définitivement le rôle des bases.

Scénario 1 — V1, répartition des requêtes

```
python3 scripts/ops.py v1
```

Le script envoie exactement vingt requêtes successives à `/info`. Avec quatre applications et le tourniquet HAProxy, le résultat attendu est cinq réponses par application, un écart relatif nul et `passed: true`.

Le terminal affiche le nom du fichier de preuve créé. Pour le montrer :

```
python3 -m json.tool "$(ls -t results/*-V1.json | head -n 1)"
```

Texte à prononcer :

> *V1 envoie vingt requêtes au répartiteur. Les quatre applications reçoivent chacune cinq requêtes. L'écart est nul et le critère accepte un écart inférieur à vingt pour cent. Cela valide la répartition lorsque les quatre serveurs sont sains.*

Scénario 2 — V2, panne d'une application

```
python3 scripts/validate-live.py v2
```

Le script génère du trafic, arrête `app1`, mesure l'éviction par HAProxy, contrôle les erreurs, puis redémarre automatiquement `app1`. La campagne finale a produit 2 939 requêtes, huit pertes pendant la transition, une période dégradée de 4,49 s et aucune erreur après cinq secondes.

Texte à prononcer :

> *Nous provoquons la panne de* `app1` *pendant que des clients continuent à consulter un résultat. Quelques requêtes peuvent échouer pendant la détection. HAProxy retire ensuite le serveur défaillant et le trafic continue sur les trois autres applications. Notre mesure finale respecte la limite de cinq secondes. Le script redémarre ensuite* `app1` *et attend son retour.*

Confirmer son retour :

```
vagrant status app1
curl -i http://192.168.56.21:8000/sante
```

Scénario 3 — R5, réplication PostgreSQL

```
python3 scripts/diagnose-db.py
python3 scripts/ops.py replication
```

Dans le diagnostic, `bdd1` doit avoir `in_recovery: false`, `bdd2` doit avoir `in_recovery: true` et le verdict doit indiquer une seule base en écriture : `bdd1`.

Le test vérifie les rôles et l'état `streaming`, écrit une valeur temporaire sur la ligne `050000`, attend qu'elle soit visible sur `bdd2`, crée une preuve, restaure la valeur d'origine puis purge le cache.

Texte à prononcer :

> *bdd1 est la seule base principale et bdd2 est en récupération. Le test refuse d'écrire si ces rôles ne correspondent pas à la topologie normale. Il modifie temporairement une ligne sur bdd1 et observe la même valeur sur bdd2. La valeur d'origine est ensuite restaurée. Cela prouve que la réplication est réellement active et pas seulement déclarée dans une configuration.*

8. Autres scénarios à connaître

V3 — montée en charge

```
python3 scripts/ops.py cache-off
python3 scripts/ops.py benchmark --label DEMO-V3
```

V3 utilise 2 000 requêtes et 50 clients simultanés, cache désactivé. Les mesures enregistrées sont 137,17 requêtes/s avec deux applications et 121,28 avec quatre, sans échec. Toutes les VM partagent le même hôte et la même base ; la baisse est donc un résultat réel à présenter honnêtement.

R2 — idempotence

```
python3 scripts/ops.py idempotence
```

Le script compare les empreintes après deux provisionnements. Le second état doit être identique au premier : `F(F(x)) = F(x)`. Cette validation prend du temps, car elle provisionne les sept VM deux fois.

E3 — cache Redis

```
python3 scripts/ops.py cache-off
python3 scripts/ops.py benchmark --label DEMO-E3-SANS-CACHE
python3 scripts/ops.py cache-on
python3 scripts/ops.py cache-purge
curl 'http://127.0.0.1:8080/resultat?matricule=000001'
python3 scripts/ops.py benchmark --label DEMO-E3-CACHE-CHAUD
```

La campagne a mesuré 121,28 requêtes/s sans cache et 1 271,20 avec la même clé en cache, soit 10,48 fois plus. C'est un meilleur cas ; le gain réel dépend des matricules répétés. Les entrées expirent après trente secondes et PostgreSQL reste la source de vérité.

E4 — déploiement sans coupure

```
python3 scripts/validate-live.py e4
```

Le script envoie du trafic et met à jour les quatre applications une par une. Chaque application est drainée, mise à jour, testée puis remise en service. La preuve finale contient 17 155 requêtes, zéro échec et les quatre applications en version 2.

V4 — panne de la base principale

Attention : exécuter V4 en dernier. Ce scénario transforme `bdd2` en base principale. Après la promotion, `bdd1` doit rester arrêtée.

```
vagrant halt bdd1
curl -i http://127.0.0.1:8080/sante
python3 scripts/ops.py promote
curl -i http://127.0.0.1:8080/sante
python3 scripts/diagnose-db.py
```

Avant la promotion, `/sante` doit répondre 503 en JSON. Après la promotion, elle revient à 200 et `bdd2` devient la seule base en écriture. **Ne jamais exécuter `vagrant up bdd1` après cette promotion.** Utiliser V5 pour restaurer la topologie.

V5 — reconstruction complète

La destruction suivie de `scripts/start.sh` au début de la vidéo constitue déjà V5. La commande automatisée officielle est :

```
time python3 scripts/ops.py rebuild --confirm-destroy-project-vms
```

Elle détruit uniquement les VM de ce projet, retire le marqueur de promotion, recrée la plateforme et attend une santé 200. La reconstruction mesurée a duré 1 042,10 s, soit environ 17 min 22 s.

V6 — répartiteur redondant

V6 ne s'applique pas. Il dépend de l'extension E1, qui n'a pas été choisie. Les extensions réalisées sont E3 et E4. Il ne faut pas prétendre que `lb1` est redondant : il reste un point unique de panne.

9. Preuves enregistrées

```
ls -1 results | tail -n 30
```

Preuve	Résultat à citer
20261018T132331048575Z-V1.json	cinq réponses par application
20261018T120512261145Z-V2-app-failure.json	huit pertes, fenêtre de 4,49 s
20261018T093618782120Z-V3-N2.txt	137,17 requêtes/s avec deux applications
20261018T115656848756Z-V3-N4.txt	121,28 requêtes/s avec quatre applications
20261018T122553087917Z-V4-total-failover.json	503, promotion, puis retour à 200
20261018T132322639742Z-V5-reconstruction.json	reconstruction en 1 042,10 s
20261018T132337157126Z-R5-replication.json	valeur observée sur bdd2
20261018T115730424215Z-E3-warm-cache.txt	1 271,20 requêtes/s avec cache chaud
20261018T122014514190Z-E4-zero-downtime.json	17 155 requêtes et zéro échec

Chaque scénario produit une preuve horodatée dans `results`*. Les essais qui ont aidé au diagnostic sont conservés, mais* `docs/VALIDATION.md` *désigne les fichiers finaux utilisés pour l'acceptation.*

10. Conclusion prête à prononcer

Cette démonstration montre que notre infrastructure est décrite par le code et reconstruite avec Vagrant. HAProxy répartit les requêtes entre quatre applications sans état et retire rapidement une application en panne. PostgreSQL réplique les données de bdd1 vers bdd2. Redis accélère les lectures répétées et le déploiement progressif met à jour les applications sans échec observé. Nos limites sont documentées : lb1 n'est pas redondant, la promotion de la base reste manuelle et toutes les VM partagent les ressources du même hôte. Les preuves et les contributions des quatre membres sont disponibles dans le dépôt GitHub. Merci.

11. Questions probables et réponses simples

Pourquoi quatre applications ?

Pour répartir la charge et continuer à répondre lorsqu'une application tombe. Elles sont identiques et sans état.

Pourquoi `/sante` interroge-t-elle PostgreSQL ?

Une application incapable de consulter la base ne rend pas le service attendu. La sonde vérifie donc une dépendance réelle et exige une base en écriture.

Pourquoi bdd2 est-elle normalement en lecture seule ?

Elle reçoit les changements de bdd1 en mode récupération. Cela évite des écritures concurrentes et réduit le risque de divergence.

Pourquoi la promotion n'est-elle pas automatique ?

Une promotion automatique sûre nécessite un consensus et une clôture fiable de l'ancienne principale. La procédure manuelle vérifie d'abord que bdd1 est arrêtée.

Redis contient-il les résultats officiels ?

Non. Redis contient des copies temporaires pendant trente secondes. PostgreSQL reste la source durable. Une panne Redis ralentit les lectures sans supprimer les résultats.

Pourquoi les performances baissent-elles avec quatre applications ?

Les applications, les bases et HAProxy partagent un seul ordinateur physique. La base et les ressources de l'hôte deviennent le goulot d'étranglement.

Comment les secrets sont-ils protégés ?

Ils sont générés localement dans `.env`, transmis seulement aux VM concernées et exclus de Git. La vidéo ne doit jamais montrer leur contenu.

Pourquoi V6 n'est-elle pas présentée ?

V6 dépend de E1, qui n'a pas été retenue. Le groupe a réalisé E3 et E4.

Est-ce que le projet fonctionne dans la VM Ubuntu ouverte manuellement ?

Non. Les commandes sont exécutées sur la machine physique. Les VM du projet sont créées automatiquement par Vagrant dans VirtualBox.

12. Incidents possibles pendant la répétition

`scripts/ops.py` est introuvable

```
cd /home/aiobi5/Documents/Codex/2026-10-18/dis/outputs/resultats-bf
```

Vagrant indique qu'une VM n'est pas démarrée

```
vagrant status
```

Si aucune promotion n'a été faite, démarrer la VM manquante avec `vagrant up NOM`. Si V4 a promu bdd2, ne pas redémarrer bdd1 ; utiliser V5.

`/sante` répond 503

```
python3 scripts/diagnose-db.py
```

Lire le verdict avant toute action. Ne pas improviser une promotion.

V1 échoue

```
vagrant status app1 app2 app3 app4
curl -i http://127.0.0.1:8080/sante
```

Attendre quelques secondes que les sondes HAProxy convergent puis relancer V1.

Le terminal affiche `^[[200~`

Appuyer sur `Ctrl+C`, revenir dans le dossier du projet et retaper la commande. Ne pas copier une sortie de commande comme s'il s'agissait d'une commande.

`/dev/vboxdrv` est absent

Arrêter la répétition, exécuter `sudo /sbin/vboxconfig` sur l'hôte, puis relancer `preflight`. Ce problème appartient à VirtualBox, pas au code du projet.

13. Ordre de sécurité à mémoriser

1. Faire V1, V2, V3, R2, R5, E3 et E4 avant V4.
2. Faire V4 en dernier, car il change le rôle de bdd2.
3. Garder bdd1 arrêtée après une promotion.
4. Utiliser V5 pour revenir à la topologie initiale.
5. Ne jamais afficher `.env`, `.runtime` ou un jeton GitHub.
6. Ne jamais annoncer que lb1 est redondant.
7. Ne jamais modifier une preuve ou inventer une mesure.

14. Liste finale avant de publier la vidéo

- ☐ Les quatre noms sont prononcés correctement.
- ☐ Les contributions Git des quatre membres sont visibles.
- ☐ Aucun secret ni adresse électronique privée n'est affiché.
- ☐ La destruction et le début de la reconstruction sont visibles.
- ☐ La fin réelle de la reconstruction et sa durée sont visibles.
- ☐ Les sept VM apparaissent comme `running`.
- ☐ `/sante`, `/info` et `/resultat` sont montrées.
- ☐ Trois scénarios sont exécutés et expliqués.
- ☐ Chaque résultat est interprété, pas seulement affiché.
- ☐ V3 est présenté honnêtement comme une baisse sur l'hôte partagé.
- ☐ Les limites de lb1 et de la promotion manuelle sont annoncées.
- ☐ La conclusion cite le dépôt GitHub et les preuves enregistrées.